МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

МЕТОДИЧНІ ВКАЗІВКИ

до виконання лабораторних робіт з дисциплін

«Сучасні Web-технології», та «Frontend-розробка»

для спеціальностей: F3 – «Комп'ютерні науки» та F1 – «Прикладна математика»

Лабораторна робота 1

Інструменти веб-розробника

Харків

НТУ «ХПІ»

2026

ЗМІСТ

ЗМІСТ 2
ВСТУП 3
Мета роботи 3
Основні задачі: 3
РОЗДІЛ 1. ТЕОРЕТИЧНІ ВІДОМОСТІ 4
1.1. Веб-браузери для розробників 4
1.2. Редактори коду 5
1.3. Онлайн-середовища розробки 7
1.3. Система контролю версій Git 8
РОЗДІЛ 2. ПРИКЛАД ВИРІШЕННЯ ЗАВДАННЯ 9
2.1. Постановка задачі 9
2.2. Покроковий алгоритм виконання 9
Завдання для самостійної роботи 16
Правила оформлення звіту про виконання лабораторної роботи 17
Контрольні питання 17
СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ 18

ВСТУП

Сучасна веб-розробка неможлива без використання спеціалізованого програмного забезпечення, яке забезпечує ефективне написання, тестування та налагодження коду. Інструментарій frontend-розробника включає широкий спектр програмних засобів: від потужних редакторів коду до хмарних середовищ розробки. Правильний вибір та володіння цими інструментами є фундаментальною навичкою, яка суттєво підвищує продуктивність роботи та якість створюваних веб-додатків.

Мета роботи

Метою лабораторної роботи є отримання практичних навичок з налаштування робочого середовища frontend-розробника, освоєння основних інструментів для написання веб-сторінок та систем контролю версій.

Основні задачі:

1. Зареєструвати обліковий запис на GitHub та створити перший репозиторій.
2. Освоїти базові команди Git для роботи з локальним репозиторієм.
3. Зареєструватися в онлайн-середовищі CodeSandbox.
4. Створити базову структуру статичного веб-сайту.
5. Встановити та налаштувати редактор коду Visual Studio Code.
6. Результати лабораторної роботи оформити у вигляді звіту зі скріншотами проміжних етапів.

РОЗДІЛ 1. ТЕОРЕТИЧНІ ВІДОМОСТІ

1.1. Веб-браузери для розробників

Вибір браузера для веб-розробки є важливим рішенням, оскільки сучасні браузери надають потужні вбудовані інструменти для тестування та налагодження веб-додатків. Найпопулярнішим серед розробників є **Google Chrome** завдяки комплексу інструментів Chrome DevTools, який дозволяє інспектувати та редагувати HTML і CSS в реальному часі, аналізувати мережеву активність, профілювати продуктивність JavaScript та проводити аудит доступності й продуктивності сайту.

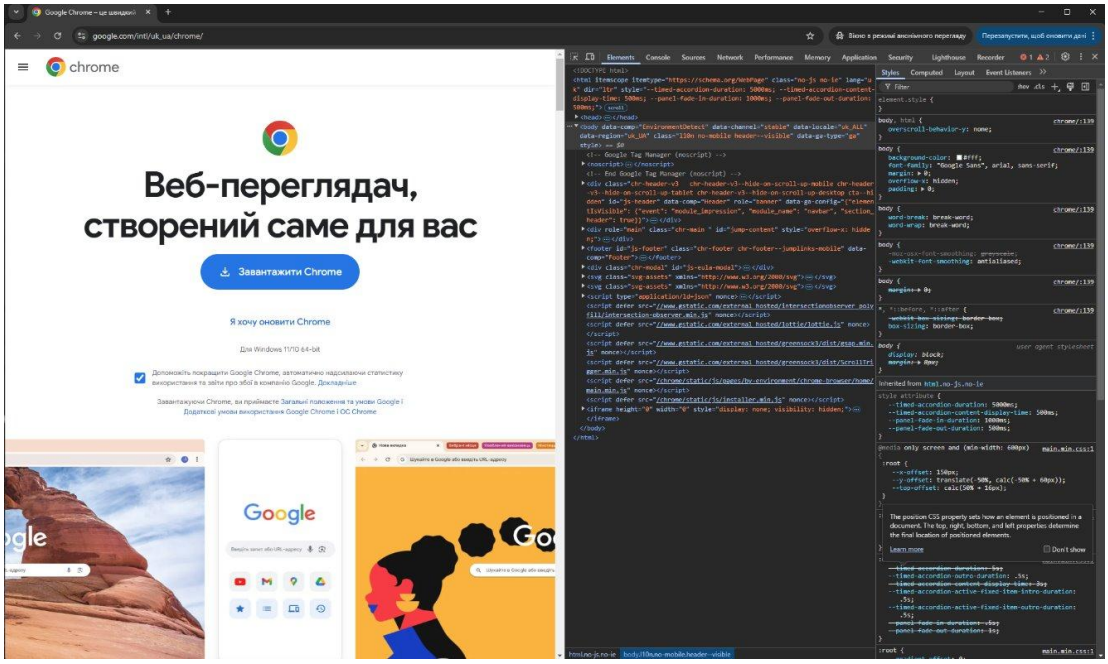

Рисунок 1. Браузер Google Chrome з інструментами Chrome DevTools.

Firefox Developer Edition є спеціалізованою версією браузера, оптимізованою для веб-розробки, яка містить найновіші функції та інструменти. Серед переваг цієї версії варто виділити потужний інспектор DOM з підтримкою flexbox та grid, розширену JavaScript консоль з аналітичними можливостями, комплексні інструменти відстеження продуктивності та вбудований режим адаптивного дизайну для тестування на різних пристроях.

Станом на кінець 2025 року Chrome залишається найпопулярнішим браузером, проте Firefox Developer Edition пропонує унікальні функції, які роблять його дуже зручним для професійної розробки. Обидва браузери підтримують розширення, що додатково розширюють їх функціональність для специфічних завдань веб-розробки.

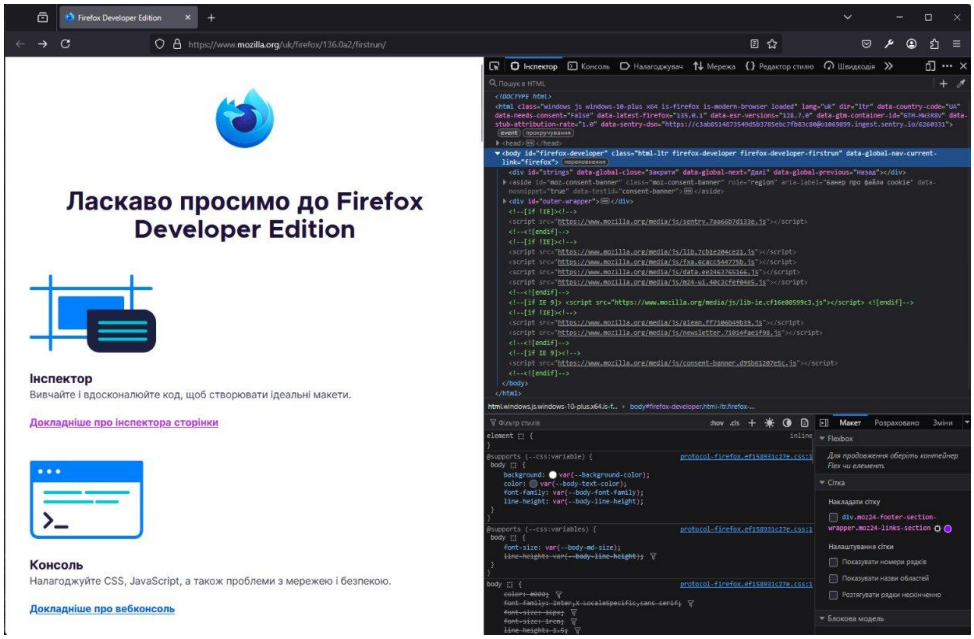

Рисунок 2. Браузер Firefox Developer Edition з інструментами DevTools

1.2. Редактори коду

Visual Studio Code є де-факто галузевим стандартом серед редакторів коду для веб-розробки. Редактор відрізняється відмінною підтримкою розширень, глибокою інтеграцією з системою контролю версій Git та потужними інструментами налагодження. Найсвіжішу версію можна завантажити посиланням: https://code.visualstudio.com/.

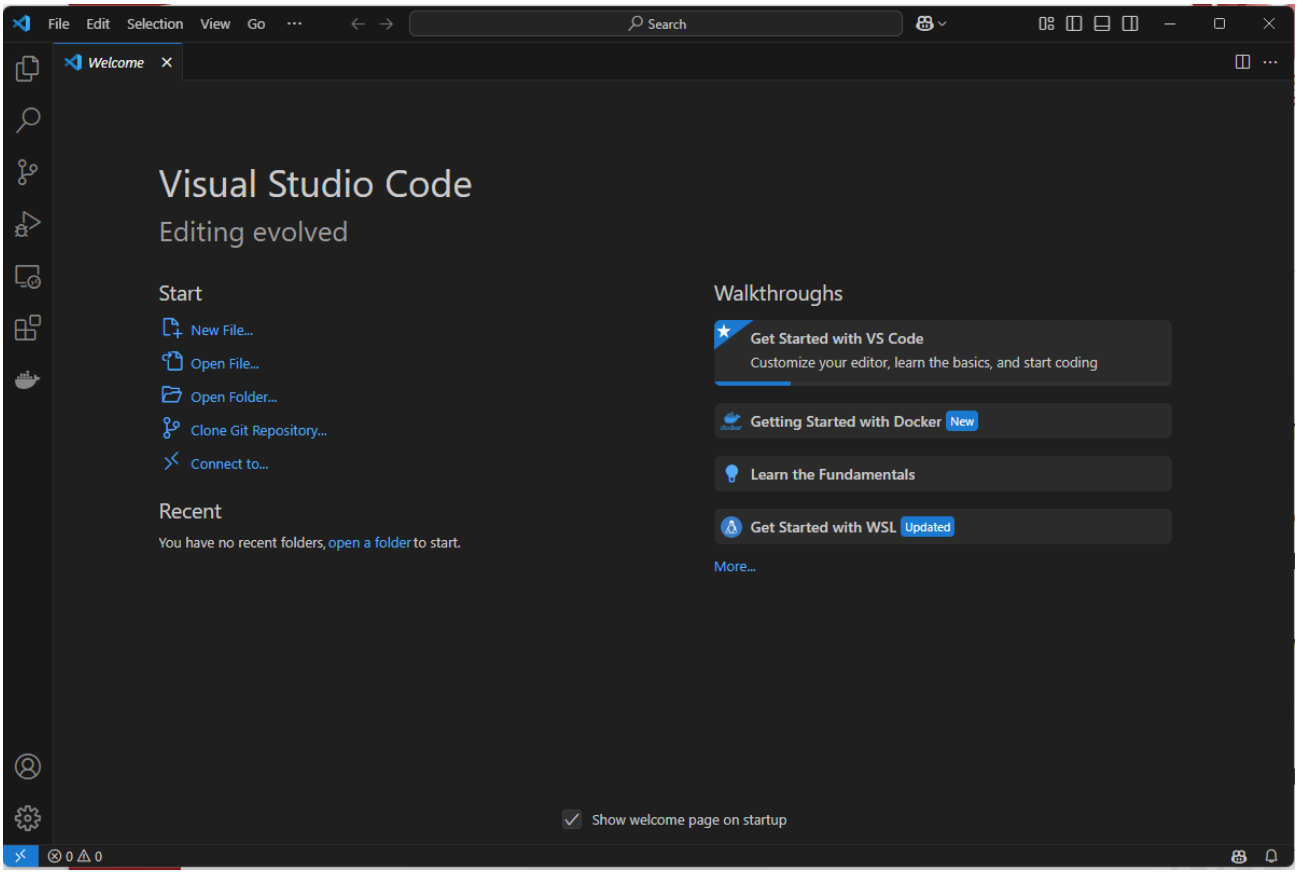

Рисунок 3. Головний екран редактора VS Code.

Вбудовані функції IntelliSense значно підвищують ефективність роботи, надаючи інтелектуальне автодоповнення коду, підказки та документацію безпосередньо в редакторі.

```
// Приклад автодоповнення у VS Code
const greeting = "Hello, World!";
console.log(greeting); // IntelliSense підкаже методи для рядків
```

WebStorm є спеціалізованим інтегрованим середовищем розробки з комплексною функціональністю для frontend-технологій. Це повноцінне IDE з великою кількістю інструментів для рефакторингу, що особливо цінно при роботі над великими проектами. WebStorm є платним програмним продуктом,

але пропонує безкоштовний пробний період для ознайомлення. Завантажити його можна за посиланням: https://www.jetbrains.com/webstorm/.

Sublime Text представляє категорію швидких редакторів з мінімалістичним інтерфейсом. Завдяки багатим можливостям налаштувань та системі керування пакетами, редактор дозволяє оптимізувати робочий процес під індивідуальні потреби розробника. Sublime Text відкриває файли миттєво, що особливо важливо при роботі з великими проектами. Більш детально можна дізнатись на офіційному сайті продукту: https://www.sublimetext.com/

1.3. Онлайн-середовища розробки

Онлайн-редактори коду є зручним рішенням для швидкого прототипування та навчання, оскільки не потребують встановлення програмного забезпечення та налаштування локального середовища. Розглянемо кілька представників нижче.

CodeSandbox (https://codesandbox.io/) є потужним середовищем для розробки веб-проектів, яке пропонує шаблони для популярних фреймворків React, Vue та Angular. Середовище має функціональність, наближену до локальних IDE, включаючи інтелектуальне автодоповнення та термінал.

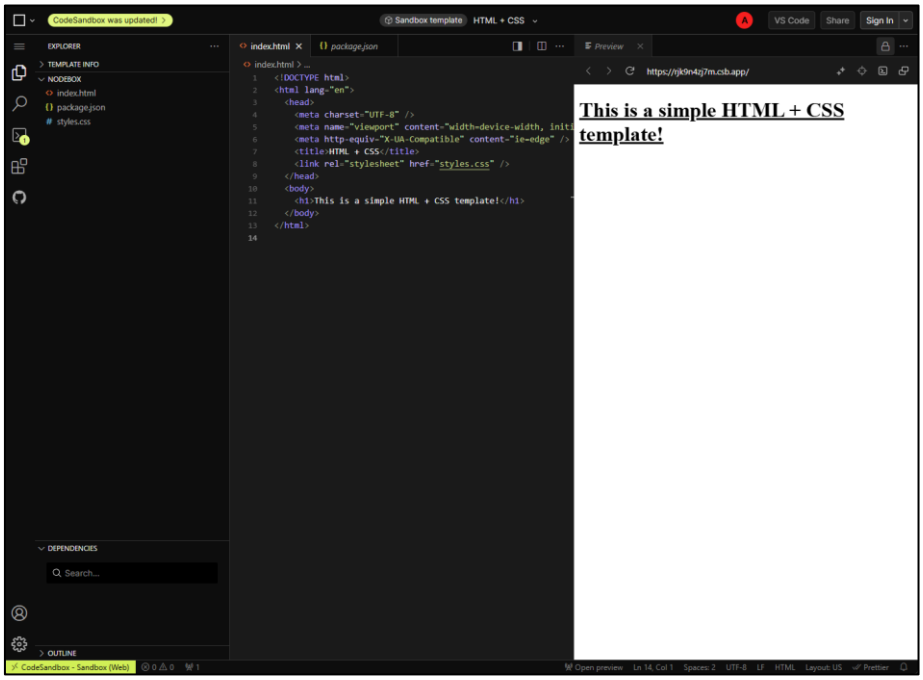

Рисунок 4. Приклад проекту в онлайн редакторі CodeSandbox.

CodePen (https://codepen.io/pen/) є спеціалізованим інструментом для швидкого прототипування, який дозволяє створювати, тестувати та демонструвати HTML, CSS та JavaScript код. Платформа підтримує підключення бібліотек та фреймворків через CDN, що спрощує експериментування з різними технологіями.

GitHub Codespaces (https://github.com/features/codespaces) забезпечує повноцінне середовище розробки у хмарі, засноване на VS Code. Це рішення гарантує безшовну інтеграцію з GitHub-репозиторіями та підтримку розширень VS Code, що дозволяє працювати над проектами з будь-якого пристрою з доступом до інтернету.

1.3. Система контролю версій Git

Git - це розподілена система контролю версій, що стала галузевим стандартом у сучасній розробці програмного забезпечення. Вона дозволяє розробникам відстежувати історію змін у файлах, повертатися до попередніх версій проекту та ефективно працювати в команді, уникаючи конфліктів при одночасному редагуванні коду. На відміну від централізованих систем, Git зберігає повну копію історії змін на локальному комп'ютері кожного розробника, що забезпечує високу швидкість роботи та можливість працювати без постійного підключення до мережі Інтернет.

Робота з Git базується на трьох основних станах файлів:

- **Working Directory** (робоча директорія),
- **Staging Area** (індекс)
- **Repository** (репозиторій).

Процес збереження змін відбувається поетапно: спочатку змінені файли додаються до індексу, а потім фіксуються у репозиторії як "коміт" (commit). Кожен коміт має унікальний ідентифікатор (хеш) та містить інформацію про автора, дату та опис змін.

Для спільної роботи та резервного копіювання використовуються віддалені репозиторії (Remote Repositories), які розміщуються на спеціалізованих хостингах, таких як **GitHub**, **GitLab** або **Bitbucket**. Синхронізація локального та віддаленого репозиторіїв дозволяє команді розробників обмінюватися кодом та інтегрувати зміни різних учасників в єдиний проект.

Для початку роботи з Git необхідно виконати ініціалізацію репозиторію командою **git init**, додати файли до індексу командою **git add** та зафіксувати зміни командою **git commit**. Підключення до віддаленого репозиторію на GitHub здійснюється командою git remote add origin [url], а відправка змін виконується командою git push.

```
# Базові команди Git
git init                    # Ініціалізація репозиторію
git add .                   # Додавання всіх файлів
git commit -m "First commit" # Створення коміту
git remote add origin https://github.com/username/repo.git
git push -u origin main     # Відправка на GitHub
```

РОЗДІЛ 2. ПРИКЛАД ВИРІШЕННЯ ЗАВДАННЯ

2.1. Постановка задачі

У рамках лабораторної роботи необхідно створити повноцінне робоче середовище frontend-розробника. Завдання включає реєстрацію облікового запису GitHub, створення локального репозиторію, його публікацію на GitHub, реєстрацію в CodeSandbox та створення базової структури статичного веб-сайту.

2.2. Покроковий алгоритм виконання

Крок 1. Реєстрація на GitHub

Перейдіть на офіційний сайт https://github.com/ та натисніть кнопку «Sign up» у верхньому правому куті.

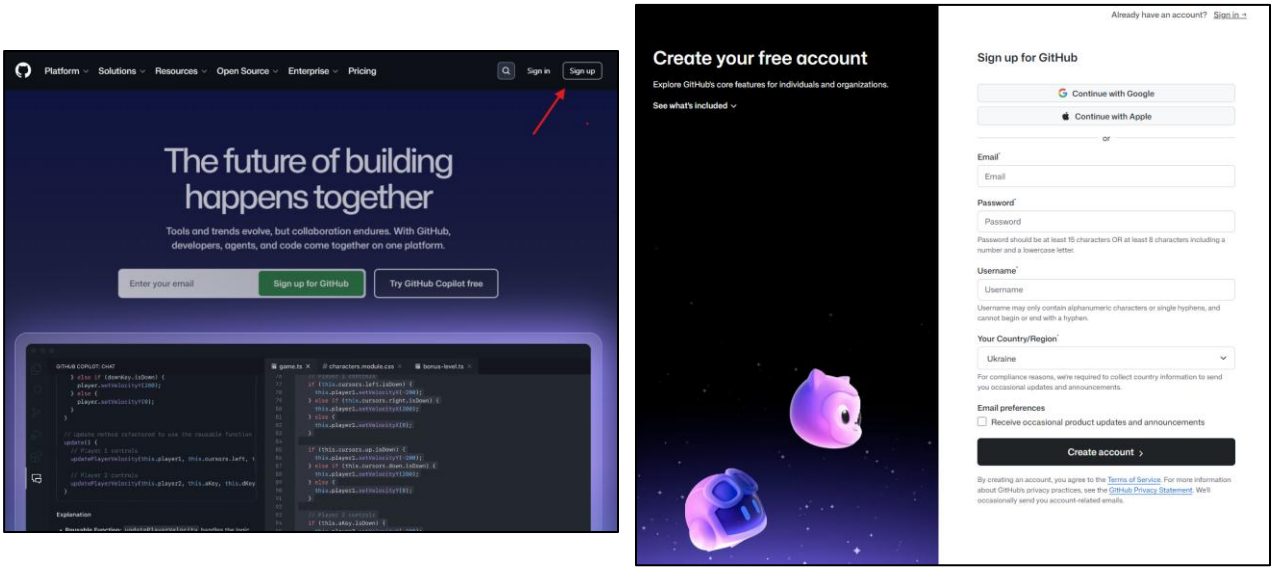

Рисунок 5. Реєстрація на github.com.

Введіть адресу електронної пошти, створіть пароль та введіть ім’я користувача. Пройдіть верифікацію та підтвердіть адресу електронної пошти. Після успішної реєстрації увійдіть у свій обліковий запис.

Крок 2. Створення нового репозиторію

На головній сторінці з репозиторіями https://github.com/repos натисніть кнопку **«New repository»**.

Введіть назву репозиторію, наприклад **«my-first-website»**, та опис за бажанням.

Виберіть тип доступу публічний або приватний.

Натисніть **«Create repository»** для створення порожнього репозиторію.

Крок 3. Ініціалізація локального репозиторію

Відкрийте термінал або командний рядок. Створіть нову папку для проекту та перейдіть до неї.

Виконайте ініціалізацію Git-репозиторію командою

```
git init
```

Налаштуйте ім’я користувача та email командами

```
git config user.name "Your Name"
```

та

```
git config user.email "your@email.com"
```

Крок 4. Створення та публікація першого коміту

Створіть файл **index.html** з базовою структурою HTML-документа.

Додайте файл до індексу командою

```
git add .
```

Створіть перший коміт командою

```
git commit -m "Initial commit"
```

Підключіть локальний репозиторій до GitHub командою

```
git remote add origin [url-вашого-репозиторію]
```

Відправте зміни на GitHub командою

```
git push -u origin main
```

Крок 5. Реєстрація в CodeSandbox

Перейдіть на сайт https://codesandbox.io/ та натисніть «Sign in» (див. рис. 6). Виберіть опцію входу через GitHub та авторизуйте доступ CodeSandbox до вашого облікового запису. Після успішної авторизації ви опинитесь на головній сторінці середовища розробки.

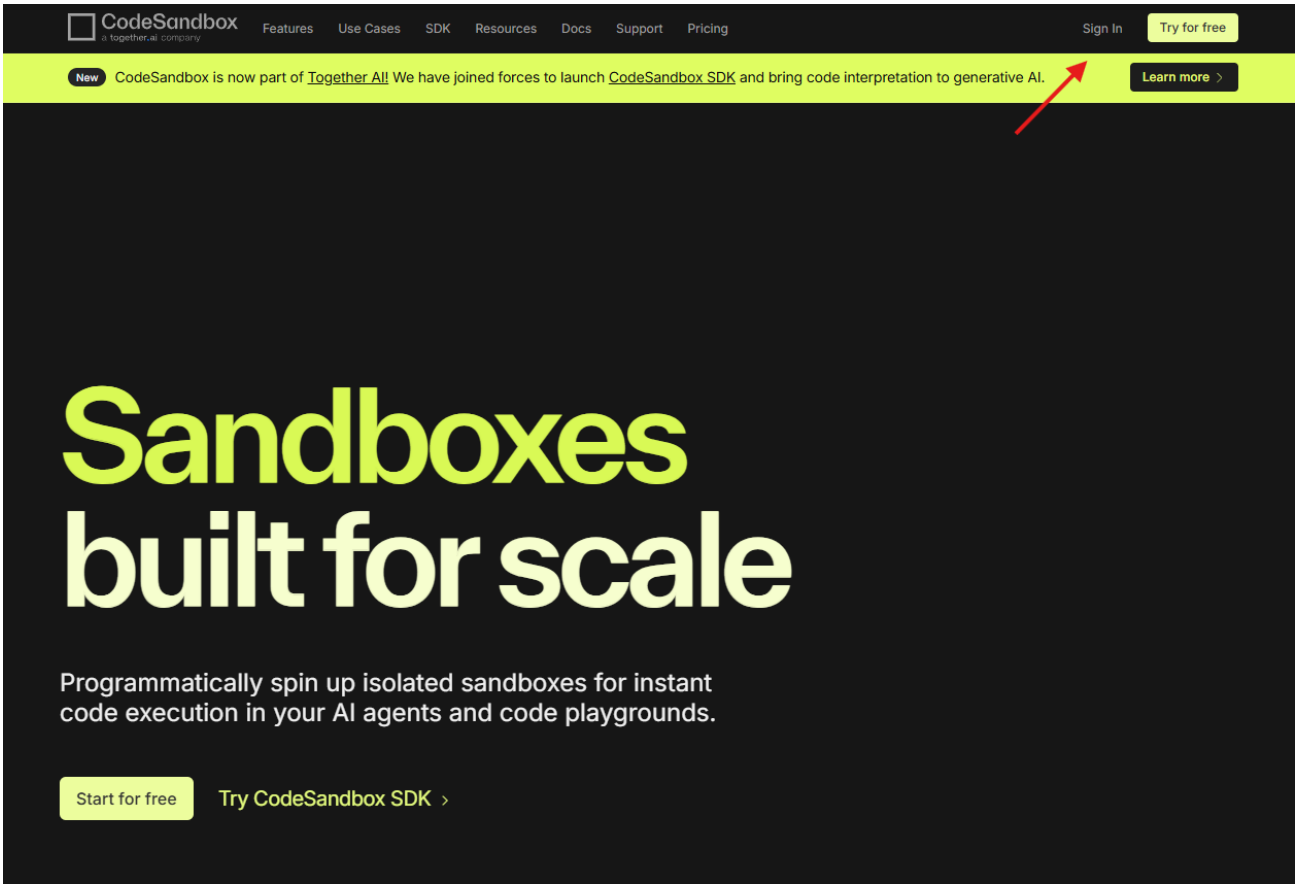

Рисунок 6. Реєстрація в CodeSandbox.

Крок 6. Створення проекту в CodeSandbox

В горі справа натисніть кнопку «+ Create».

У розділі «Start with a template» виберіть шаблон «HTML + CSS» (див. рис. 7).

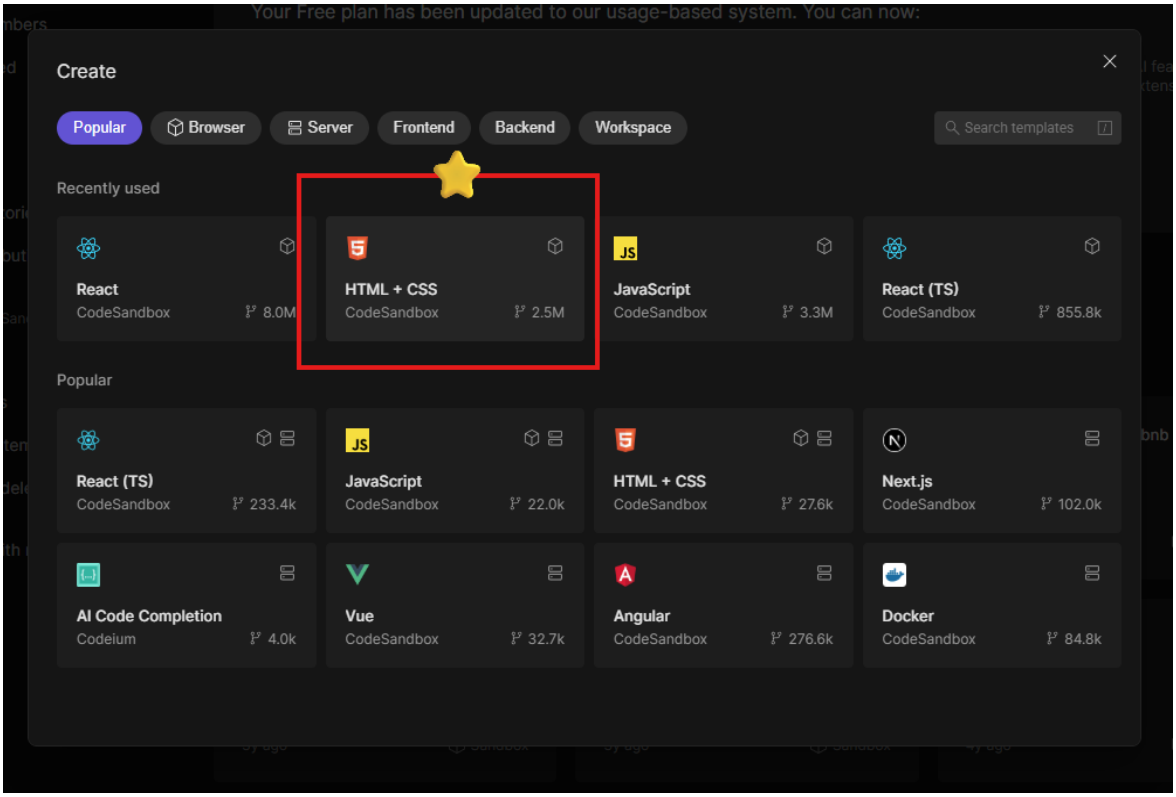

Рисунок 7. Вибір шаблону проекту.

Дочекайтеся завантаження шаблону та перегляньте структуру проекту в бічній панелі редактора.

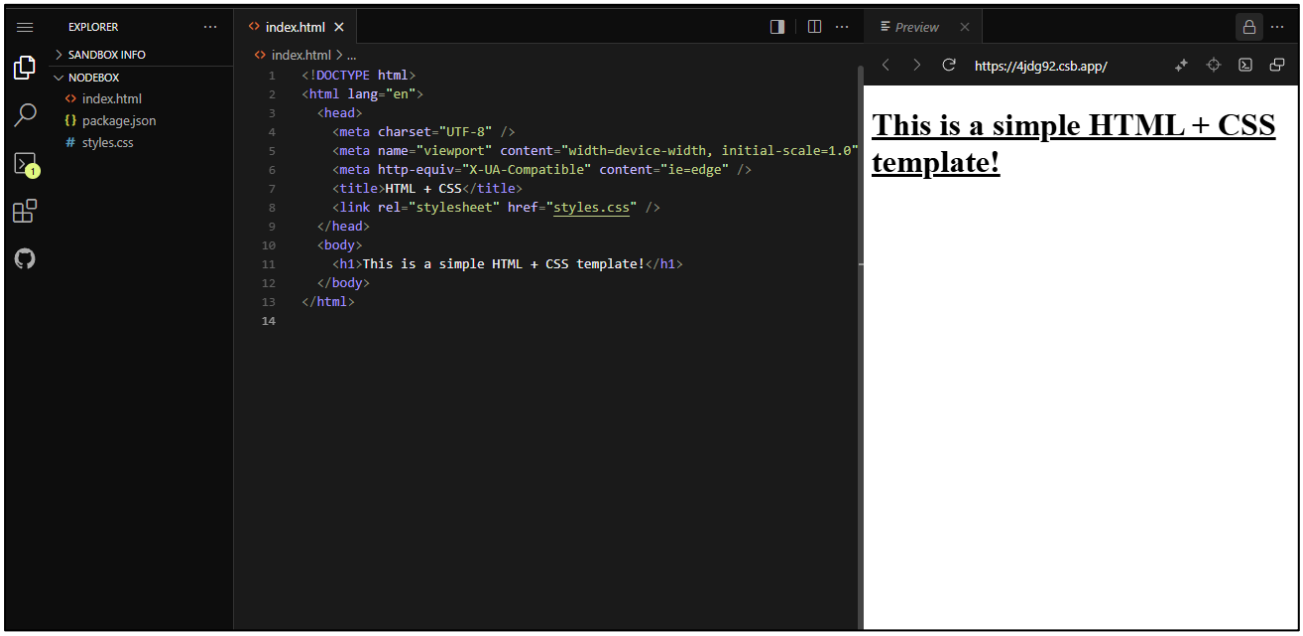

Рисунок 8. Створений пустий проект в CodeSandbox.

Крок 7. Створення структури директорій

У CodeSandbox натисніть правою кнопкою миші на корінь проекту та виберіть «New Folder».

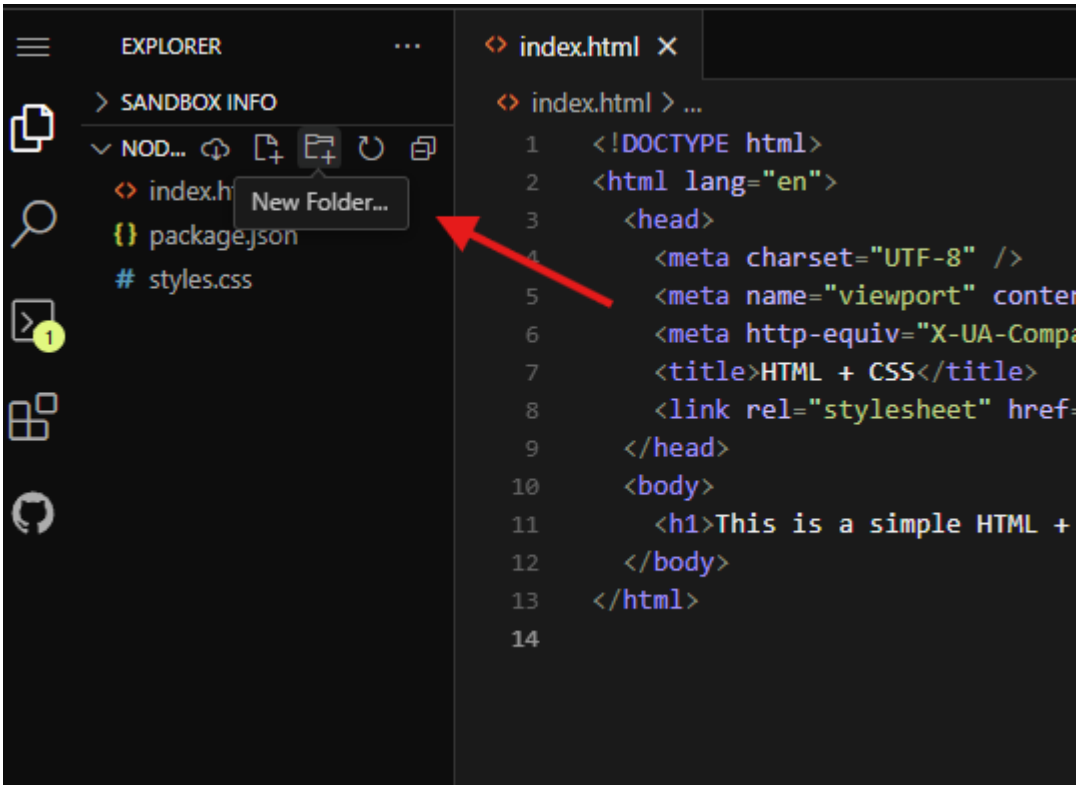

Рисунок 9. Кнопка створення директорій

Створіть наступні директорії: **css, js, images**.

Для створення файлів натисніть правою кнопкою миші на потрібну директорію та виберіть «New File».

Створіть файли:

- index.html,
- about.html у папці pages,
- style.css у папці css,
- script.js у папці js.

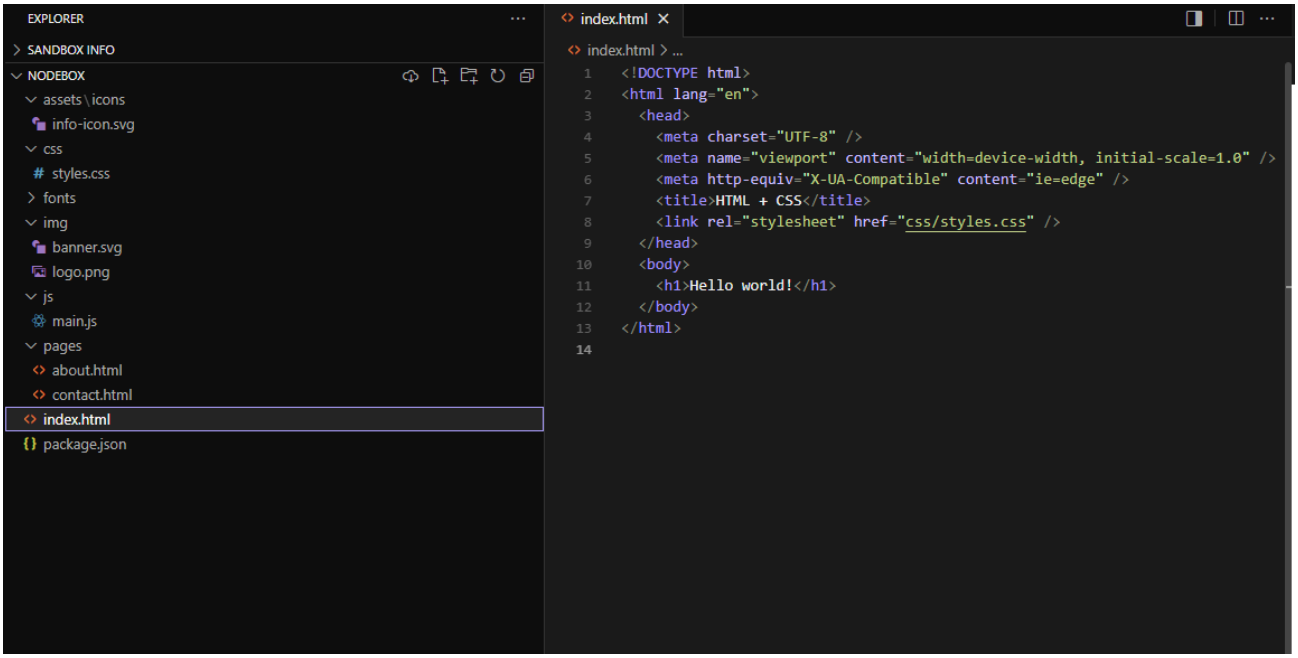

Рисунок 10. Створений проект в CodeSandbox

Крок 8. Збереження та перегляд проекту

CodeSandbox автоматично зберігає зміни. Перегляньте результат у правій панелі, де відображається попередній перегляд веб-сторінки. Для публікації проекту натисніть «Publish» та виберіть опцію публікації.

Крок 9. Фіналізація роботи

Перевірте роботу всіх створених файлів у браузері. Переконайтеся, що структура директорій відповідає вимогам. Збережіть посилання на опублікований проект для включення до звіту. Переконайтеся, що репозиторій на GitHub містить усі необхідні файли.

Завдання для самостійної роботи

Додатково виконайте і додайте до звіту щонайменше 2 приклади відповідно до наведених нижче варіантів.

Номер варіанту	Номер завдання з переліку нижче	
Варіант 1	1	2
Варіант 2	1	3
Варіант 3	1	4
Варіант 4	5	6
Варіант 5	5	7
Варіант 6	5	8

1. **Основи VS Code.** Встановіть Visual Studio Code на власний комп'ютер. Налаштуйте п'ять корисних розширень для frontend-розробки та опишіть їх призначення.

2. **Робота з Git.** Створіть новий репозиторій на GitHub. Клонуйте його на локальний комп'ютер. Створіть три коміти з різними змінами та відправте їх на віддалений репозиторій.

3. **Структура сайту.** Створіть повноцінну структуру багатосторінкового сайту: головна сторінка, сторінка «Про нас», сторінка контактів, сторінка послуг. Використовуйте CodeSandbox.

4. **Firefox DevTools.** Встановіть Firefox Developer Edition. Відкрийте DevTools (F12). Знайдіть та опишіть три інструменти, яких немає в Chrome.

5. **CodePen.** Зареєструйтесь на CodePen. Створіть демонстрацію з використанням HTML, CSS та JavaScript. Опублікуйте її та надайте посилання.

6. **Зведення інструментів.** Створіть документ, що порівнює три онлайн-середовища розробки (CodeSandbox, CodePen, StackBlitz). Вкажіть переваги та недоліки кожного.

7. **Автоматизація.** Створіть файл README.md для вашого проекту з інструкцією щодо встановлення та запуску.

8. **Дослідження плагінів.** Встановіть п’ять плагінів VS Code, пов’язаних з Git. Опишіть функціональність кожного та як вони покращують робочий процес.

Правила оформлення звіту про виконання лабораторної роботи

Звіт про виконання лабораторної роботи повинен містити:

1. Титульний аркуш.
2. Мету роботи.
3. Результати виконання роботи:
    - виконане перше покрокове завдання, що складається з покрокового налаштування середовищ розробки зі скрінами проміжних етапів;
    - виконане одне з наведених вище варіантів зі скрінами проміжних етапів;
4. Розгорнуті висновки по виконаній роботі з зазначенням можливих складнощів.

Всі копії екранів повинні бути оформлені як рисунки з відповідними підписами під ними (наприклад: Рисунок 1.1 – Копія екрану з результатами роботи програми). Рисунки слід оформляти на зразок тих, що наведені в даній лабораторній роботі. На кожен з рисунків повинно бути посилання в тексті звіту.

Контрольні питання

1. Які переваги надає використання Chrome DevTools порівняно з іншими браузерами?

2. Чому Visual Studio Code є найпопулярнішим редактором коду серед frontend-розробників?
3. Опишіть різницю між редактором коду та інтегрованим середовищем розробки (IDE).
4. Які переваги надає використання онлайн-середовищ розробки, таких як CodeSandbox?
5. Які базові команди Git необхідно знати для початку роботи з системою контролю версій?
6. Чому важливо використовувати систему контролю версій при роботі над проектом?
7. Які типи файлів слід додавати до файлу .gitignore у веб-проектах?
8. Чим відрізняється публічний репозиторій від приватного на GitHub?
9. Які переваги надає GitHub Codespaces порівняно з локальним середовищем розробки?
10. Як підключити віддалений репозиторій до локального?
11. Яка різниця між командами git add, git commit та git push?
12. Як вирішити конфлікт злиття (merge conflict) при роботі з Git?
13. Які альтернативи GitHub існують для хостингу Git-репозиторіїв?
14. Як налаштувати автоматичний деплой сайту на GitHub Pages?

СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ

1. Бородкіна І. Л. Web-технології та web-дизайн: застосування мови HTML для створення електронних ресурсів : навч. посіб. / І. Л. Бородкіна, Г. О. Бородкін. – Київ : Ліра-К, 2020. – 210 с.
2. Haverbeke M. Eloquent JavaScript: A Modern Introduction to Programming / M. Haverbeke. – 3rd Edition. – No Starch Press, 2018. – 480 p.

3. MDN Web Docs. Resources for developers, by developers [Електронний ресурс]. – Режим доступу: https://developer.mozilla.org/

4. W3C Official Page [Електронний ресурс]. – Режим доступу: http://www.w3.org/

5. HTML Підручник [Електронний ресурс]. – Режим доступу: https://w3schoolsua.github.io/html/index.html

6. Visual Studio Code Documentation [Електронний ресурс]. – Режим доступу: https://code.visualstudio.com/docs

7. Git Handbook. GitHub Docs [Електронний ресурс]. – Режим доступу: https://docs.github.com/en/get-started/using-git/about-git

8. Chrome DevTools Documentation [Електронний ресурс]. – Режим доступу: https://developer.chrome.com/docs/devtools

9. CodeSandbox Documentation [Електронний ресурс]. – Режим доступу: https://codesandbox.io/docs

10. Pro Git Book [Електронний ресурс] / S. Chacon, B. Straub. – 2nd Edition. – Режим доступу: https://git-scm.com/book/en/v2

11. Frontend Developer Handbook 2024 [Електронний ресурс]. – Режим доступу: https://frontendmasters.com/guides/learning-roadmap/